PROGRAM

Client_side

```
#include<stdio.h>
#include<stdlib.h>
#include<netinet/in.h>
#include<sys/types.h>
#include<sys/socket.h>
#include<string.h>
#include<unistd.h>
#include<time.h>
int main()
{
	int client_sock;
	char recv_msg[256];
	int n,arr[10][10];
	client_sock=socket(AF_INET,SOCK_STREAM,0);
	struct sockaddr_in server_addr;
	server_addr.sin_family=AF_INET;
	server_addr.sin_port=htons(4533);
	server_addr.sin_addr.s_addr=INADDR_ANY;
	int con=connect(client_sock,(struct sockaddr*)&server_addr,sizeof(server_addr));
	if(con==-1)
	{
		printf("Error in establishing connection...\n");
		return 1;
	}
	recv(client_sock,recv_msg,sizeof(recv_msg),0);
	printf("%s\n",recv_msg);
```

```
        printf("Enter the size of matrix:");
        scanf("%d",&n);
        send(client_sock,&n,sizeof(n),0);
        srand(time(0));
        printf("The matrix is:\n");
        for(int i=0;i<n;i++)
        {
                for(int j=0;j<n;j++)
                {
                        arr[i][j]=rand()%50;
                        printf("%d ",arr[i][j]);
                }
               printf("\n");
        }
        send(client_sock,arr,sizeof(arr),0);
        char mat_result[50];
        recv(client_sock,mat_result,sizeof(mat_result),0);
        printf("The result from server\n%s\n",mat_result);
        close(client_sock);
        return 0;
}
```

Server_side

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<netinet/in.h>
#include<sys/types.h>
```

```
#include<unistd.h>
#include<sys/socket.h>
int check_upper(int mat[10][10],int m){
	for(int i=0;i<m;i++)
	{
		for(int j=0;j<i;j++)
		{
		if(mat[i][j]!=0)
			return 0;
		}
	}
	return 1;
}
int check_lower(int mat[10][10],int m){
	for(int i=0;i<m;i++){
		for(int j=i+1;j<m;j++){
			if(mat[i][j]!=0)
				return 0;
		}
	}
	return 1;
}
int check_diagonal(int mat[10][10],int m)
{
	for(int i=0;i<m;i++)
	{
		for(int j=0;j<m;j++)
		{
```

```
                        if(i!=j && mat[i][j]!=0)
                                return 0;
                }
        }
        return 1;
}

int main(){
        int server_sock;
        char server_msg[]="You have reached the server...\n";
        server_sock=socket(AF_INET,SOCK_STREAM,0);
        struct sockaddr_in server_addr;
        server_addr.sin_family=AF_INET;
        server_addr.sin_port=htons(4533);
        server_addr.sin_addr.s_addr=INADDR_ANY;
        bind(server_sock,(struct sockaddr*)&server_addr,sizeof(server_addr));
        listen(server_sock,5);
        int client_sock=accept(server_sock,NULL,NULL);
        send(client_sock,server_msg,sizeof(server_msg),0);
        int n;
        recv(client_sock,&n,sizeof(n),0);
        printf("The size of matrix: %d\n",n);
        int arr[10][10];
        recv(client_sock,arr,sizeof(arr),0);
        printf("The matrix recieved from client:\n");
        for(int i=0;i<n;i++){
                for(int j=0;j<n;j++){
                        printf("%d ",arr[i][j]);
```

```
            }
             printf("\n");
        }
        int upper,lower,diagonal;
        upper=check_upper(arr,n);
        lower=check_lower(arr,n);
        diagonal=check_diagonal(arr,n);
        char mat_msg[50];
        if(diagonal==1)
        {
            strcpy(mat_msg,"Diagonal");
        }
        else if(lower==1)
        {
            strcpy(mat_msg,"Lower Traingular");
        }
        else if(upper==1)
        {
            strcpy(mat_msg,"Upper Triangular");
        }
        else
        {
            strcpy(mat_msg,"None of the mentioned");
        }
        send(client_sock,mat_msg,sizeof(mat_msg),0);
        printf("The matrix is analysed and the result is given to the client...\n");
        return 0;
}
```

OUTPUT

Client_side:

[2024cse333@localhost cn]$ gcc clientmat.c -o clnmat

[2024cse333@localhost cn]$ ./clnmat

You have reached the server...

Enter the size of matrix:3

The matrix is:

1 2 1

0 0 1

3 4 3

The result from server

None of the mentioned

Server_side:

[2024cse333@localhost cn]$ gcc servermat.c -o srvmat

[2024cse333@localhost cn]$ ./srvmat

The size of matrix: 3

The matrix recieved from client:

1 2 1

0 0 1

3 4 3

The matrix is analysed and the result is given to the client...

UPPER-

Server_side:

The size of matrix: 3

The matrix recieved from client:

1 2 3

0 2 4

0 0 3

The matrix is analysed and the result is given to the client...

Client_side:

You have reached the server...

Enter the size of matrix:3

Enter the matrix:1 2 3 0 2 4 0 0 3

The result from server

Upper Triangular

LOWER-

Server_side:

The size of matrix: 3

The matrix recieved from client:

1 0 0

1 2 0

1 2 3

The matrix is analysed and the result is given to the client...

Client_side:

You have reached the server...

Enter the size of matrix:3

Enter the matrix/n:1 0 0 1 2 0 1 2 3

The result from server

Lower Traingular

PROGRAM

Server side

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<unistd.h>
#include<netinet/in.h>
#define port 4533
#define size 1024
void translate(char str[])
{
        char temp[1024] = "";
        char *word;
        const char *delimiters=" ,!?-.";
        word = strtok(str,delimiters);
        while(word != NULL)
        {
                if(strcmp(word, "idc") == 0)
                        strcat(temp, "I don't care ");
                else if(strcmp(word, "irl") == 0)
                        strcat(temp, "in real life ");
                else if(strcmp(word, "atm") == 0)
                        strcat(temp, "at the moment ");
                else if(strcmp(word, "tbf") == 0)
                        strcat(temp, "to be fair ");
                else if(strcmp(word, "tbh") == 0)
                        strcat(temp, "to be honest ");
                else if(strcmp(word, "ig") == 0)
```

```
                strcat(temp, "I guess ");
        else
        {
                strcat(temp, word);
                strcat(temp, " ");
        }
        word = strtok(NULL,delimiters);
    }
    strcpy(str, temp);
}

int main()
{
    int server_sock;
    char buffer[size];
    struct sockaddr_in server,client;
    socklen_t len=sizeof(client);
    server_sock=socket(AF_INET,SOCK_DGRAM,0);
    server.sin_family=AF_INET;
    server.sin_port=htons(port);
    server.sin_addr.s_addr=INADDR_ANY;
    bind(server_sock,(struct sockaddr*)&server,sizeof(server));
    printf("\nServer is running..\n");
    recvfrom(server_sock,buffer,sizeof(buffer),0,(struct sockaddr*)&client,&len);
    printf("Received:\n%s\n",buffer);
    translate(buffer);
    sendto(server_sock,buffer,sizeof(buffer),0,(struct sockaddr*)&client,len);
    printf("\nTranslated message sent..\n");
```

```
        close(server_sock);
        return 0;
}
```

Client side

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<unistd.h>
#include<netinet/in.h>
#define port 4533
#define size 1024
int main()
{
        int client_sock;
        struct sockaddr_in server;
        char buffer[size];
        client_sock=socket(AF_INET,SOCK_DGRAM,0);
        server.sin_family=AF_INET;
        server.sin_port=htons(port);
        server.sin_addr.s_addr=INADDR_ANY;
        printf("\nClient is running..\n");
        printf("Enter the sentence:");
        fgets(buffer,sizeof(buffer),stdin);
        sendto(client_sock,buffer,sizeof(buffer),0,(struct sockaddr*)&server,sizeof(server));
        printf("\nSentence sent to server..\n");
        recvfrom(client_sock,buffer,sizeof(buffer),0,NULL,NULL);
        printf("\nTranslated sentence\n%s\n",buffer);
```

```
        close(client_sock);
        return 0;
}
```

OUTPUT

Server_side:

```
 [2024cse333@localhost cn]$ ./udp_s
Server is running..
Received:
Really idc about the server. But irl I had to do the project atm. ig I am doing this tbf and tbh to the professor.
Translated message sent..
```

Client_side:

```
[2024cse333@localhost cn]$ ./udp_c
Client is running..
Enter the sentence:Really idc about the server. But irl I had to do the project atm. ig I am doing this tbf and tbh to the professor.
Sentence sent to server..
Translated sentence
Really I don't care about the server But in real life I had to do the project at the moment I guess I am doing this to be fair and to be honest to the professor
```